

Объектно-ориентированное программирование

Никита Остроухов

Кафедра информатики МФТИ

Взаимодействие с пользователем

```
int main()  
{
```

```
    Ball balls[100];
```

```
    ...
```

```
}
```

Добавим возможность пользователю
определять **количество** газовых частиц в
идеальном газе

Взаимодействие с пользователем

```
#include <iostream>

int main()
{
    int count = 0;

    std::cout << "Enter count:" << std::endl;
    std::cin >> count;

    Ball balls[100];

    ...
}
```

Потоки **std::cout** и **std::cin** используются для
вывода вывода

Взаимодействие с пользователем

```
#include <iostream>

int main()
{
    int count = 0;

    std::cout << "Enter count:" << std::endl;
    std::cin >> count;

    Ball balls[100];

    ...
}
```



Библиотека для ввода вывода

Взаимодействие с пользователем

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int count = 0;
```

```
    std::cout << "Enter count:" << std::endl;
```

```
    std::cin >> count;
```

```
    Ball balls[100];
```

```
    ...
```

```
}
```



std::endl переводит текст на новую строку и синхронизирует буфер обмена

Взаимодействие с пользователем

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int count = 0;
```

```
    std::cout << "Enter count:" << std::endl;
```

```
    std::cin >> count;
```

```
    Ball balls[count];
```

```
    ...
```

```
}
```

Ошибка
компиляции



Компилятор должен знать число элементов в **статическом массиве** во время компиляции

Статическая и динамическая память

Статическая память

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int count = 0;
```

```
    std::cout << "Enter count:" << std::endl;
```

```
    std::cin >> count;
```

```
    Ball balls[count];
```

```
    ...
```

```
}
```



**Статическое
выделение памяти**

Статическая память

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int count = 0;
```

```
    std::cout << "Enter count:" << std::endl;  
    std::cin >> count;
```

```
    Ball balls[count];
```

```
    ...
```

```
}
```

Создана в функции и
освободится после
выхода из функции

Статическая память

- Статически выделенная память **освобождается самостоятельно** и существует только в своей области видимости

Статическая память

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int count = 0;
```

```
    std::cout << "Enter count:" << std::endl;  
    std::cin >> count;
```

```
    Ball balls[count];
```

```
    ...
```

```
}
```

Статическая память

- Статически выделенная память **освобождается самостоятельно** и существует только в своей области видимости
- Создается и освобождается **быстро** - **без** использования **аллокатора** памяти

Статическая память

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int count = 0;
```

```
    std::cout << "Enter count:" << std::endl;  
    std::cin >> count;
```

```
    Ball balls[count];
```

```
    ...
```

```
}
```

Статическая память

- Статически выделенная память **освобождается самостоятельно** и существует только в своей области видимости
- Создается и освобождается **быстро** - **без** использования **аллокатора** памяти
- Требуется знание размера массива в момент компиляции

Динамическая память

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int count = 0;
```

```
    std::cout << "Enter count:" << std::endl;
```

```
    std::cin >> count;
```

```
    Ball* balls = new Ball;
```

```
    ...
```

```
}
```

Динамическое
выделение



Динамическая память

Динамическая память

```
#include <iostream>

int main()
{
    int count = 0;

    std::cout << "Enter count:" << std::endl;
    std::cin >> count;

    Ball* balls = new Ball[count];

    ...
}
```

Динамическая память

- Выделяется в **runtime** приложения и не требует знания размера в момент компиляции

Динамическая память

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int count = 0;
```

```
    std::cout << "Enter count:" << std::endl;  
    std::cin >> count;
```

```
    Ball* balls = new Ball[count];
```

```
    ...
```

```
    delete[] balls;
```

```
}
```

Освобождение памяти



Динамическая память

- Выделяется в **runtime** приложения и не требует знания размера в момент компиляции
- Освобождается **только вручную** что является и плюсом и минусом

Динамическая память

```
#include <iostream>

int main()
{
    int count = 0;

    std::cout << "Enter count:" << std::endl;
    std::cin >> count;

    Ball* balls = new Ball[count];

    ...

    delete[] balls;
}
```

Динамическая память

- Выделяется в **runtime** приложения и не требует знания размера в момент компиляции
- Освобождается **только вручную** что является и плюсом и минусом
- Вызов аллокатора памяти долгий

Ошибки при работе с памятью

Мертвая память

```
Ball* initArray()
{
    Ball balls[100];
    return balls;
}

int main()
{
    balls = initArray();
    balls[0] = ...
}
```

Какая здесь проблема?

Мертвая память

```
Ball* initArray()
{
    Ball balls[100];
    return balls;
}

int main()
{
    balls = initArray();
    balls[0] = ...
}
```



Переменная **balls** выделена **статически**
и является **локальной** переменной **initArray**

Мертвая память

```
Ball* initArray()  
{  
    Ball balls[100];  
    return balls;  
}  
  
int main()  
{  
    balls = initArray();  
    balls[0] = ...  
}
```



Память выделенная под balls **освободится**
после окончания функции **initArray**

Мертвая память

```
Ball* initArray()  
{  
    Ball balls[100];  
    return balls;  
}  
  
int main()  
{  
    balls = initArray();  
    balls[0] = ...  
}
```



Мы пользуемся “**мертвой**” памятью

Мертвая память

```
Ball* initArray()  
{  
    Ball balls[100];  
    return balls;  
}
```

```
int main()  
{  
    balls = initArray();  
    balls[0] = ...  
}
```

↖

**В незанятый участок памяти
кто угодно может что-то
писать и читать**

Мы пользуемся “**мертвой**” памятью

Утечка памяти

```
Ball* initArray(int len)
{
    Ball* balls = new Ball[len];
    return balls;
}
```

```
int main()
{
    balls = initArray(100);
    balls = initArray(200);

    delete[] balls;

    ...
}
```

Какая здесь проблема?

Утечка памяти

```
Ball* initArray(int len)
{
    Ball* balls = new Ball[len];
    return balls;
}

int main()
{
    balls = initArray(100);
    balls = initArray(200);

    delete[] balls;

    ...
}
```



Освободится только **массив** из 200 элементов, а массив из 100 элементов не освободится - мы **потеряли** его указатель

Утечка памяти

```
Ball* initArray(int len)
{
    Ball* balls = new Ball[len];
    return balls;
}
```

```
int main()
{
    balls = initArray(100);
    balls = initArray(200);

    delete[] balls;

    ...
}
```

Перезатирает указатель

Освободится только **массив** из 200 элементов, а массив из 100 элементов не освободится - мы **потеряли** его указатель

Мертвая память

```
struct SmartArray {  
    SmartArray(int size) {  
        array = new char[size];  
    }  
  
    Ball* array;  
};  
  
int main() {  
    SmartArray array1(100);  
    delete[] array1.array;  
}
```

Мертвая память

```
struct SmartArray {  
    SmartArray(int size) {  
        array = new char[size];  
    }
```

```
    ~SmartArray(int size) {  
        delete[] array;  
    }
```

```
    char* array;  
    int length;
```

```
};
```

```
int main() {  
    SmartArray array1(100);  
}
```

Деструктор



Чтобы не освобождать руками
мы написали **деструктор**

Мертвая память

```
struct SmartArray {  
    SmartArray(int size) {  
        array = new Ball[size];  
    }
```

```
    ~SmartArray(int size) {  
        delete[] array;  
    }
```

```
    Ball* array;  
    int length;
```

```
};
```

```
int main() {  
    SmartArray array1(100);  
}
```

Деструктор



Деструктор вызывается каждый раз при уничтожении переменной

Мертвая память

```
void moveAll(SmartArray all)
{
    for (int i = 0; i < all.length; i++)
        all.array[i].move();
}

int main() {
    SmartArray array1 = SmartArray(100);
    moveAll(array1);

    array1.array[0] = ...
}
```

Вызов функции **moveAll** приводит к проблеме мертвой памяти

Мертвая память

```
void moveAll(SmartArray all)
{
    for (int i = 0; i < all.length; i++)
        all.array[i].move();
}

int main() {
    SmartArray array1 = SmartArray(100);
    moveAll(array1);

    array1.array[0] = ...
}
```

Обращение к освобожденной
памяти



Вызов функции **moveAll** приводит к
проблеме мертвой памяти

Мертвая память

```
void moveAll(SmartArray all)
{
    for (int i = 0; i < all.length; i++)
        all.array[i].move();
}

int main() {
    SmartArray array1 = SmartArray(100);
    moveAll(array1);
    array1.array[0] = ...
}
```



Переменная array1 копируется в функцию
moveAll

Мертвая память

```
void moveAll(SmartArray all)
{
    for (int i = 0; i < all.length; i++)
        all.array[i].move();
}
```



```
int main() {
    SmartArray array1 = SmartArray(100);
    moveAll(array1);

    array1.array[0] = ...
}
```

Аргумент содержит все **те же поля** что и локальная переменная **main**

В том числе поле **array**

Мертвая память

```
void moveAll(SmartArray all)
{
    for (int i = 0; i < all.length; i++)
        all.array[i].move();
}

int main() {
    SmartArray array1 = SmartArray(100);
    moveAll(array1);

    array1.array[0] = ...
}
```



По окончании функции **moveAll**
переменная **all** освобождается

Мертвая память

```
void moveAll(SmartArray all)
{
    for (int i = 0; i < all.length; i++)
        all.array[i].move();
}

int main() {
    SmartArray array1 = SmartArray(100);
    moveAll(array1);

    array1.array[0] = ...
}
```



То есть вызовется **деструктор** SmartArray
который сделает delete[] array

Мертвая память

```
void moveAll(SmartArray all)
{
    for (int i = 0; i < all.length; i++)
        all.array[i].move();
}

int main() {
    SmartArray array1 = SmartArray(100);
    moveAll(array1);

    array1.array[0] = ...
}
```



Здесь **array** поле уже освобождено
аллокатором памяти

**Динамическая память
требует ответственности**

Задание

Сделать возможность в идеальном газе задавать произвольное количество шаров